

Faculté d'Informatique

Université des Sciences et de la Technologie Houari Boumediene

Cycle Ingéniorat– 1ère Année

Rapport Technique

Application Gaza Cipher

Présenté par : CHIKH Hafsa, AITBENAMARA Cerine

Encadré par : MrZAIDI

Année universitaire 2024-2025

1 Introduction

L’application **Gaza Cipher** est une solution de messagerie sécurisée conçue pour garantir la confidentialité des échanges grâce à des techniques de cryptographie modernes. Inspirée par des outils tels que Signal et Proton-Mail, elle se distingue par une interface utilisateur élégante aux couleurs pastel, une navigation intuitive via deux onglets (envoi et réception), et une flexibilité dans le choix des méthodes de chiffrement. Gaza Cipher permet aux utilisateurs de sélectionner entre un mode de chiffrement automatique (système) ou manuel (utilisateur), supportant les algorithmes RSA, AES, César, et Vigenère. Toutes les opérations sont réalisées localement, renforçant la sécurité des données.

Ce projet, développé dans le cadre du cycle ingénieur à l’USTHB, illustre notre capacité à intégrer des concepts avancés de cybersécurité dans une application fonctionnelle et esthétiquement plaisante.

2 Objectifs du projet

- L’application Gaza Cipher poursuit les objectifs suivants :
- Développer une messagerie chiffrée de bout en bout avec une interface utilisateur moderne et attrayante.
 - Offrir une flexibilité dans le chiffrement via des modes système (aléatoire) et utilisateur (personnalisé).
 - Séparer les fonctionnalités d’envoi (chiffrement) et de réception (déchiffrement) pour une meilleure clarté.
 - Garantir une gestion sécurisée des utilisateurs, des clés, et des messages.
 - Maintenir une architecture locale pour éliminer les vecteurs d’attaque externes.

3 Fonctionnalités principales

3.1 Gestion sécurisée des utilisateurs

- **Création de compte** : Hachage des mots de passe avec SHA-256 pour protéger contre les attaques par table arc-en-ciel.
- **Authentification** : Vérification locale des identifiants.
- **Génération de clés** : Création automatique de paires de clés RSA (2048 bits) lors de l’inscription.
- **Déconnexion** : Option de déconnexion pour permettre le changement d’utilisateur sans relancer l’application.

3.2 Chiffrement de bout en bout

Gaza Cipher propose quatre algorithmes de chiffrement, chacun avec des options système et utilisateur (voir Tableau III).

TABLE 1 – Algorithmes de chiffrement et options		
Algorithme	Mode Système	Mode Utilisateur
RSA	Clé publique du destinataire	Clé publique du destinataire
AES	Clé aléatoire (128 bits)	Clé de 16 caractères
César	Décalage aléatoire (1–25)	Décalage choisi (1–25)
Vigenère	Mot-clé aléatoire (6 lettres)	Mot-clé choisi (lettres)

- **Mode système** : Sélection aléatoire de l’algorithme et des paramètres (ex. : décalage pour César, mot-clé pour Vigenère).
- **Mode utilisateur** : Personnalisation des paramètres (ex. : choix du décalage pour César, saisie d’une clé AES).
- **Processus** : Les messages sont chiffrés avant envoi, utilisant la clé publique du destinataire pour RSA.

3.3 Déchiffrement explicite

- **Onglet réception** : Affiche les messages reçus dans une liste avec l’algorithme utilisé.
- **Bouton « Déchiffrer sélectionné »** : Déchiffre le message sélectionné à l’aide de la clé privée (RSA) ou des paramètres stockés.
- **Affichage** : Le texte clair est présenté dans une fenêtre contextuelle pour une démonstration claire.

3.4 Interface utilisateur

- **Design pastel** : Palette de couleurs douces (lavande, menthe, rose pastel) pour une esthétique moderne.
- **Navigation à deux onglets** : Onglets « Envoi » et « Réception » pour séparer les actions de chiffrement et de déchiffrement.
- **Flexibilité** : Menu déroulant pour choisir l’algorithme et le mode de chiffrement.

4 Technologies et outils

- **Langage** : Python 3.6+.
- **Interface graphique** : Tkinter avec le thème `clam` pour un rendu pastel professionnel.
- **Cryptographie** :
 - PyCryptodome pour RSA (PKCS1_OAEP) et AES (mode EAX).
 - Implémentations personnalisées pour César et Vigenère.
- **Stockage** : Fichiers JSON (`users.json`, `messages.json`) et PEM (`keys/`) pour les clés RSA.

5 Architecture et structure

La structure du projet est organisée comme suit :

```
gaza_cipher/  
  main.py           % Point d'entrée de l'application  
  data/  
    users.json      % Données utilisateur hachées  
    messages.json   % Messages chiffrés avec métadonnées  
    keys/           % Clés RSA privées et publiques  
  README.md        % Instructions d'installation et d'utilisation
```

6 Sécurité : Une priorité absolue

- **Hachage des mots de passe** : SHA-256 pour une protection robuste.
- **Chiffrement sécurisé** :
 - RSA avec padding PKCS1_OAEP pour une sécurité asymétrique.
 - AES avec mode EAX pour une authentification intégrée.
 - César et Vigenère avec paramètres stockés localement.
- **Stockage cloisonné** : Séparation des clés, utilisateurs, et messages dans des fichiers dédiés.
- **Localisation** : Toutes les opérations sont effectuées localement, éliminant les risques liés aux serveurs distants.

7 Conclusion

Gaza Cipher est une application de messagerie sécurisée qui allie une interface pastel élégante, une navigation intuitive à deux onglets, et une cryptographie avancée. Les modes système et utilisateur offrent une flexibilité

unique, permettant un chiffrement automatique ou personnalisé. Ce projet démontre notre expertise en cybersécurité, notre capacité à concevoir une interface utilisateur moderne, et notre engagement à produire une solution fonctionnelle et professionnelle. Gaza Cipher répond aux exigences académiques tout en offrant une expérience utilisateur agréable et sécurisée.